

实验二 熟悉服务器环境

一、实验目的

- 1、熟悉 Docker swarm 的构建
- 2、掌握用基于 Maven 的项目在服务器上编译打包的过程
- 3、掌握华为 SWR 的使用

二、实验环境

三台云服务器，配置为 1G 内存，2 核 CPU。操作系统为 ubuntu 18.04

三、实验内容及要求

1、安装 docker

在所有的服务器都需要安装 docker

先更新 ubuntu 的安装源

```
apt-get update
```

再安装 docker

```
apt-get install docker.io
```

2. 安装 jdk17 和 maven

选择一台服务器作为编译用服务器。安装 jdk17 和 maven

安装 JDK17

用 `apt-get install openjdk-17-jdk` 安装 jdk17

安装 maven

用 `wget https://dlcdn.apache.org/maven/maven-3/3.8.8/binaries/apache-maven-3.8.8-bin.zip` 下载 maven 3.8.8

用 `unzip apache-maven-3.8.8-bin.zip` 将 maven 展开到本地目录

在 .profile 文件最后增加环境变量

```
export PATH=MAVEN_HOME/bin:$PATH
```

 其中 MAVEN_HOME 是 maven 展开的目录

用 `source .profile` 让环境变量生效

用 `mvn -version` 看 maven 的版本和 jdk 的版本是否正确

3. docker swarm 初始化

如果在 docker swarm 上部署，需要先安装 docker swarm，参考以下网页 <https://docs.docker.com/engine/swarm/swarm-tutorial/create-swarm/> 选择一台机器作为管理节点，运行

```
docker swarm init
```

将 `docker swarm join --token ....` 拷贝到其他的工作节点运行

如果忘记上述命令可以用 `docker swarm join-token worker` 重新获得上述命令
各节点加入后可以用 `docker node ls` 看到各节点的状况

创建 docker swarm overlay network

用 `docker network create --driver overlay my-net` 在 docker swarm 内部建立一个虚拟网络，未来所有的服务都在这个虚拟网络里。

4. 安装 MySQL 镜像和服务

更新目标服务器的 label

为目标服务器定义 label，方便 docker swarm 在创建 Service 时，将 Service 部署在目标服务器上，以下我们在 node1 上定义了一个 label `server=mysql`

```
docker node update --label-add server=mysql node1
```

在 docker swarm 中创建服务

MySQL 的配置文件目录 `conf.d` 和数据库初始化脚本都在 `productdemo` 的 `mysql` 目录下，需要把这些文件拷贝到运行 `mysql` 的节点上，并映射到容器中。同时在 `root` 目录建立 `mysql/log` 和 `mysql/data` 分别用来存放 `mysql` 的日志和数据库文件

用 `chmod a+wr mysql/log` 将日志目录赋予所有人读写的权限，`mysql/data` 目录也做相同处理

用以下命令创建 `mysql`

```
docker service create --name mysql --with-registry-auth --constraint  
node.labels.server==mysql --publish published=3306,target=3306 --  
mount
```

```
type=bind,src=/root/JavaEEPlatform/5.MyBatis/productdemo/mysql/sql,dst=/sql,readonly --mount
```

```
type=bind,src=/root/JavaEEPlatform/5.MyBatis/productdemo/mysql/conf.d,dst=/etc/mysql/conf.d,readonly --mount
```

```
type=bind,src=/root/mysql/log,dst=/var/log/mysql --mount
```

```
type=bind,src=/root/mysql/data,dst=/var/lib/mysql --network my-net -e  
MYSQL_ROOT_PASSWORD=123456 -d swr.cn-north-
```

```
4.myhuaweicloud.com/oomall-javaee/mysql:latest
```

其中 `--with-registry-auth` 是带 token 访问私有镜像，如果访问官方公开镜

像无需此参数

`--mount type=bind,src=/root/OOMALL/mysql/sql,dst=/sql,readonly` 是将 OOMALL 的数据 SQL 脚本 mount 到容器中

`--mount`

`type=bind,src=/root/OOMALL/mysql/conf.d,dst=/etc/mysql/conf.d,readonly` 是将 OOMALL 中的 MySQL 设置 mount 到容器中

`--mount type=bind,src=/root/mysql/log,dst=/var/log/mysql` 是将操作系统的 `/root/mysql/log` mount 到容器中的 MySQL 日志目录，这样在操作系统里就能看到 mysql 的日志

`--mount type=bind,src=/root/mysql/data,dst=/var/lib/mysql` 是将操作系统的 `/root/mysql/data` mount 到容器中的 MySQL 数据目录，这样将数据库数据存储到操作系统的目录下

`-e MYSQL_ROOT_PASSWORD=123456` 是设定数据库 root 账户密码

如果要将 mysql 的端口暴露出来 加上 `--publish`

`published=3306,target=3306`

`swr.cn-north-4.myhuaweicloud.com/oomall-javaee/mysql:latest` 是 mysql 在华为云上的私有镜像，如果访问官网镜像请改成 `mysql:latest`

在运行 mysql 服务的节点上运行 sql 脚本

看一下 mysql 的服务运行在哪台服务器

`docker service ps mysql`

切换到运行 mysql 服务的机器，看一下 mysql 容器在这台机器的 container id，将容器的 CONTAINER ID 拷贝替换下述命令中 [CONTAINER ID]，用这个容器运行 mysql 的命令

`docker exec -it [CONTAINER ID] mysql -uroot -p`

用 root 账号登录 mysql 服务器，在运行起来的 mysql 命令行中用 `source /sql/database.sql` 建立 productdemo 数据库

初始化数据

用 `use oomall_demo` 切换数据库

用 `source /sql/product.sql` 插入初始数据

5. 编译打包 ProductMyBatis

在安装了 maven 和 jdk 的节点上编译打包 ProductMyBatis

编译 ProductMyBatis

将代码从服务器上克隆下来，`git clone https://gitee.com/mingqcn/spring-fundamentals.git`

在 4. MyBatis/ProductDemoMyBatis 目录下运行

`mvn clean pre-integration-test -Dmaven.test.skip=true`

在编译完成后用 `docker images` 能看到镜像已经在本地存在

上传镜像到华为 swr

将镜像名称改成华为 swr 所需要的名称，其中组织名称是在华为 swr 上建立的组织

```
docker tag {镜像名称}:{版本名称} swr.cn-north-4.myhuaweicloud.com/{组织名称}/{镜像名称}:{版本名称}
```

将镜像上传到华为 swr `docker push swr.cn-north-4.myhuaweicloud.com/{组织名称}/{镜像名称}:{版本名称}`，上传前应该建立本节点和华为 swr 的临时或永久登录关系。

6. 部署 ProductMyBatis 服务

在部署的节点（如 node2）的 /root/ 上建立 logs 目录。

在管理机上创建服务

```
docker node update --label-add server=goods node2
```

```
docker service create --name productmybatis --with-registry-auth --network my-net --constraint node.labels.server==goods --publish published=8080,target=8080 --mount
```

```
type=bind,source=/root/logs,destination=/app/logs -d swr.cn-north-4.myhuaweicloud.com/{组织名称}/{镜像名称}:{版本名称}
```

用 `docker service ls` 看所有服务运行状况

`docker service ps productmybatis` 看 productmybatis 服务的具体状况

在部署的节点上的 logs 目录用 `tail -f productmybatis.log` 看日志文件的内容

四、实验报告及提交要求

本实验是个人实验，无需提交实验报告。由助教现场检查上述实验内容是否完成。